# U7280249 COMP1130 Assignment 1 - Technical Report

## Introduction

NOTE: THE 'D' KEY IS NOT BINDED TO PRINTING TO CONSOLE, INSTEAD USE 'E'

The program that was implemented as part of this assignment works by running the command `cabal v2-run shapes` in the console. Upon doing this, the user is prompted with a URL that takes them to a webpage titled ‘Blank Canvas’. The program responds to the following keys/mouse actions:

| **Key:** | **Effect:** |
|---|---|
| `Esc` | Clear the Canvas |
| `M` | Display the mystery Image |
| `C` | Cycle through Colour options |
| `T` | Cycle through tool options |
| `Backspace/Delete` | Remove the last drawn shape |
| `[_]` | Finish drawing a polygon if the user has clicked more than three times and is using the polygon tool |
| `XY` | Flips the entire canvas about the axis specified by the key |
| `WASD` | Translates a selected area by one unit up, left, down, and right respectively |
| `Q` | Deletes all shapes in the selected range |
| `E` | Prints current canvas to terminal |
| | |
| **Mouse Action:** | |
| Click-drag-release | Creates a shape/selected area for all tools other than the polygon tool and parallelogram tool |
| Click and release | Adds a point for polygon/parallelogram creation, remove the selected area |

After pressing ‘T’ while in square mode, will change the program to select mode, not line mode. Attempting to press backspace while there is a selected range will delete the last drawn shape along with the selected area.

## Task 1

A mistake was made in implementing part A as if the tool was in use, and the user pressed the ‘T’ key, it would clear the tool of all inputs. This was fixed after submission of part A.

All helper functions in task 1 were tested using the `cabal v2-test` command. This was done along with a direct analysis of the code to ensure that the function produced a desired output.

Due to the simplicity of the functions, only one input was provided via GHCi, which was to show that the function worked as intended. This involved manual input:

- For `toolToLabel` and `nextTool`, I inputted `LineTool Nothing`.
- For `nextColour`, I inputted black.

## Task 2

The most interesting problem in task 2 was translating the user-inputted points into drawable shapes. I decided to use helper functions to create a set of points that corresponded to a polygon.

For Parallelograms, I used vectors to find the fourth point: Given vectors $\overrightarrow{OB}, \overrightarrow{OA}, \overrightarrow{OC}$ (in order of provided arguments), $\overrightarrow{OD} = \overrightarrow{OB} + \overrightarrow{OC} - \overrightarrow{OA}$. This required the use of a `vecAdd` function.

For Circles, the radius was half the distance between the provided points. I translated the circle according to the midpoint.

For Squares, all four corners lie on a line that is intersects one of, and is perpendicular to the line between, the two midpoints. The corner is separated from the midpoint by half the length of the square. Using trigonometry, all four points can be found.

For all other shapes, the way to draw them was trivial.

To test this, I used the command
`CodeWorld.drawingOf (View.colourShapesToPicture Model.mystery)`
in the GHCi simulation.

## Task 3

I used helper functions that break down the task into smaller problems:

For the `PointerPress` event, I:

1. Checked if it was a `ParallelogramTool` with two arguments
2. If not, it would 'append' a new point to the Tool.
3. If it was, it would construct a parallelogram using `toolToShape`.

For the `PointerRelease` event:

1. If it was a `ParallelogramTool` or `PolygonTool`, it would return the model unchanged.
2. Otherwise, the program would construct a shape using `toolToShape` and the argument of the tool was reset using `nullTool`.

For the Spacebar, the program used `toolToShape` if the current tool was a `polygonTool` with at least three points. For the backspace, I removed the head of the list, if the list was not empty.

For the `isIncomplete` function, I assumed that the first argument wasn't `Nothing` if the second argument wasn't `Nothing`.

To test, I drew one of each shape, cycling through every colour and tool type, and then deleted them from the canvas. I also got a non-COMP1130 student to try and break the program.

## Task 4

I used recursion in order to flip the coordinates of each point in `[ColourShape]`.

To simplify the helper functions, I included a boolean condition that would determine whether to flip on the X or Y axis.

To test this, I pressed 'X' and 'Y' on the mystery image, in different combinations.

## Task 5.1 and 5.3

I introduced a new tool: `SelectTool`. Which would act similar to the rectangle tool.

I made a function that would return true if a given `ColourShape` is in range. Since there is no API function determining if a Picture encloses another, I manually programmed one:

- For most shapes, it was trivial.
- For parallelograms and squares, I had to detect if any corner points were out of range.
- For Circles, I found that there was no way to tell if it lied within region for every possible circle. The best I could do was to inscribe a square in the circle, then do as above.

Working with a top-down approach I made two functions, which would:

1. Go through every element in `[ColourShape]`.
2. If it was in range, it would either remove the shape from the list, or move all of it's points by a unit vector.

Lastly, I bound these functions to a ‘Q’ key for delete, the ‘WASD’ keys for translation, and a click-drag-release for selection. Note that after every keypress, I removed the selection area.

It was not too difficult implementing this component, however the number of functions I required to cover every case was a lot. However, I don't think there is any method that is easier.

To test, I inputted values into every function on the GHCi simulator, and checked the return. I did this multiple times for any boundary conditions. E.g., I tested `pointInRange` multiple times for points in the x-bounds, the y-bounds, both at once, on the rectangle’s border etc.

The selected range was highlighted with a grey box, which made it distinct for the user to see. When the selection tool was deactivated, there would be no grey box.

## Task 6 (Technical Report)

In terms of writing this document, I found it easy. What was hard was staying below the word limit of 1500, as a result I was required to be concise and to leave out unnecessary details.

## Overall Reflection

In terms of the implementation approach, using a modular top-down approach prevented too many repeated operations in code via polymorphism. E.g., using `VecAdd` for both parallelograms and translations or `SelectTool` for both task 5.1 and 5.3.

Most assumptions were made in task 5, where I assumed the user would only need to translate a shape once or twice, and that it would be very unlikely for a circle to be flagged as in-range when it was not.

Most functions were tested using a range of black-box and white-box methods. Certain testing resources were already provided for black-box testing i.e., the Mystery image and the `cabal v2-test` command.

Edge cases were only of concern in task 5, where there were many conditional statements in use.

Overall, it was more beneficial time-wise to try break the program on a system level.

I didn't encounter any serious technical issues other than the inability for the program to detect a circle in range, *for all* circles. To solve this, the code would have to inscribe a polygon with infinite points. All other technical/conceptual issues could be solved with an understanding of vectors, co-ordinate geometry, and trigonometry.

The resultant program is simple enough that the user would be able to understand what they could do in seconds. The use of commenting makes the source code understandable, however, due to Haskell not being stated/dynamic/imperative, it's limited in explaining algorithmic operation. The internal documentation of the code is decent, but there could be improvements for some of the helper functions.

Further conceptual issues came in understanding what the assignment was asking me to do. This was part of why I made the error in part A - a misinterpretation/misreading of the requirements.